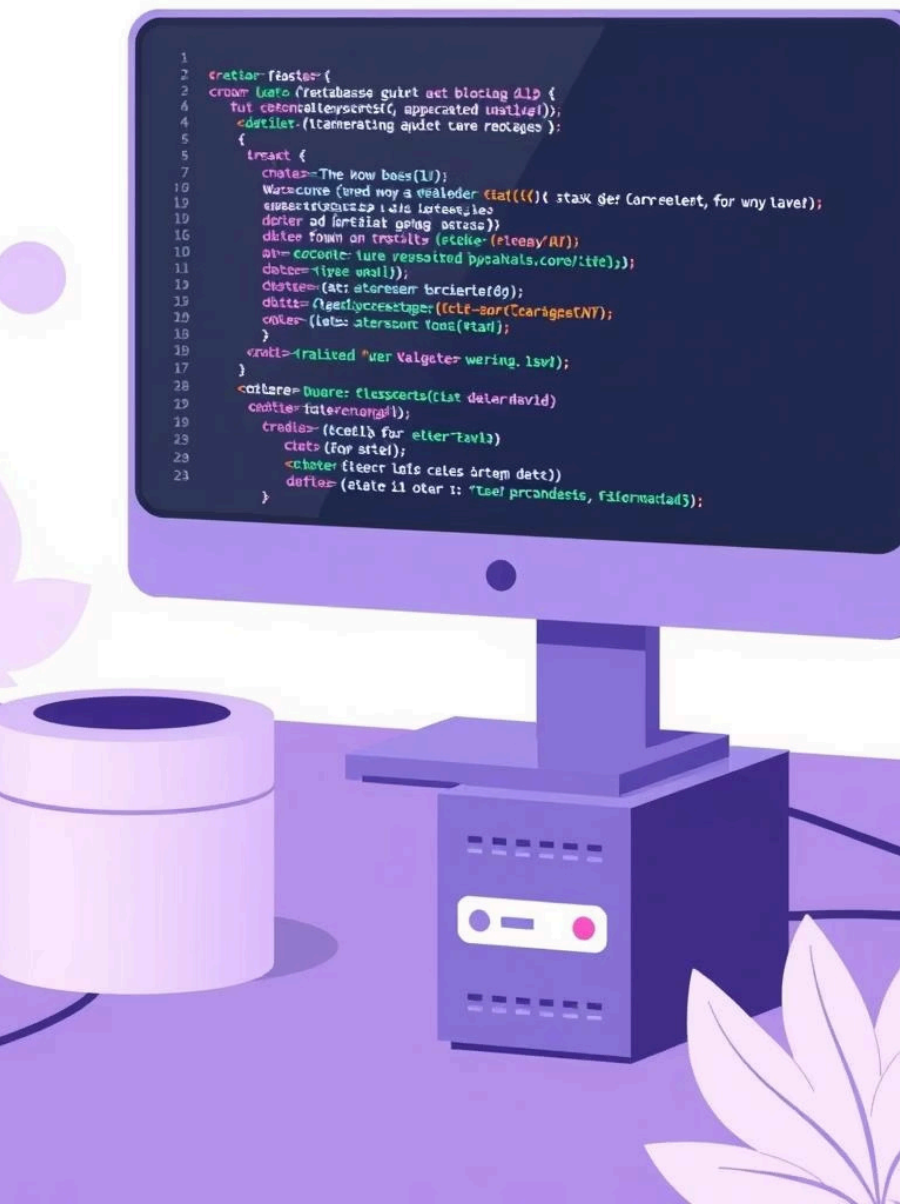




```
1
2 <entity name="User" {
3   <id name="id" type="int" {
4     <generator name="id-generator" type="sequence" {
5       <sequence name="id-sequence" {
6         <start name="start" value="1"/>
7         <increment name="increment" value="1"/>
8       }
9     }
10  }
11  <string name="name" type="string" {
12    <length name="length" value="255"/>
13  }
14  <string name="email" type="string" {
15    <length name="length" value="255"/>
16  }
17  <string name="password" type="string" {
18    <length name="length" value="255"/>
19  }
20  <string name="phone" type="string" {
21    <length name="length" value="20"/>
22  }
23 }
```

Ciclo de Vida de una Entidad en JPA

De objeto Java a registro en la base... y viceversa

Introducción al Ciclo de Vida

- Es la serie de estados por los que pasa una entidad desde su creación hasta su eliminación.
- Comprenderlo evita errores comunes y optimiza el rendimiento.
- El `EntityManager` es clave en esta gestión.

Los 4 Estados Principales

Las entidades de JPA transitan por distintos estados, cada uno con un comportamiento específico.

1

Nuevo (Transient)

Objeto recién creado, fuera de la gestión de JPA.

2

Gestionado (Persistent)

Controlado activamente por el `EntityManager`.

3

Desasociado (Detached)

Tiene ID, pero ya no está bajo la gestión activa.

4

Eliminado (Removed)

Marcado para ser borrado de la base de datos.

Estado: Nuevo (Transient)

Una entidad está en estado **Nuevo** cuando ha sido instanciada, pero aún no se ha asociado a un contexto de persistencia de JPA.

Características

- Recién creado con `new`.
- No tiene ID asignado por la base de datos.
- JPA aún no lo "conoce".

Ejemplo

```
Producto producto = new Producto("Laptop", 1200.0);  
// 'producto' está en estado Nuevo
```

Estado: Gestionado (Persistent)

Una entidad **Gestionada** es aquella que está bajo el control directo del `EntityManager`.

Acciones que lo gestionan

- `persist()`: Guarda una nueva entidad.
- `find()`: Recupera una entidad existente.
- Consultas JPQL/Criteria API.

Beneficios clave

- **Dirty Checking**: Los cambios se sincronizan automáticamente.
- **Caché de Primer Nivel**: Mejora el rendimiento de lecturas.

Estado: Desasociado (Detached)

Una entidad **Desasociada** es una entidad Gestionada que ha sido "desconectada" del `EntityManager`.

Características

- Tiene un ID de base de datos.
- No hay `EntityManager` activo para ella.
- Los cambios **no se sincronizarán automáticamente**.

Cómo se llega

- Cierre del `EntityManager`.
- Uso explícito de `detach()`.
- Serialización de la entidad.

Estado: Eliminado (Removed)

Una entidad **Eliminada** está marcada para ser borrada de la base de datos.

Proceso

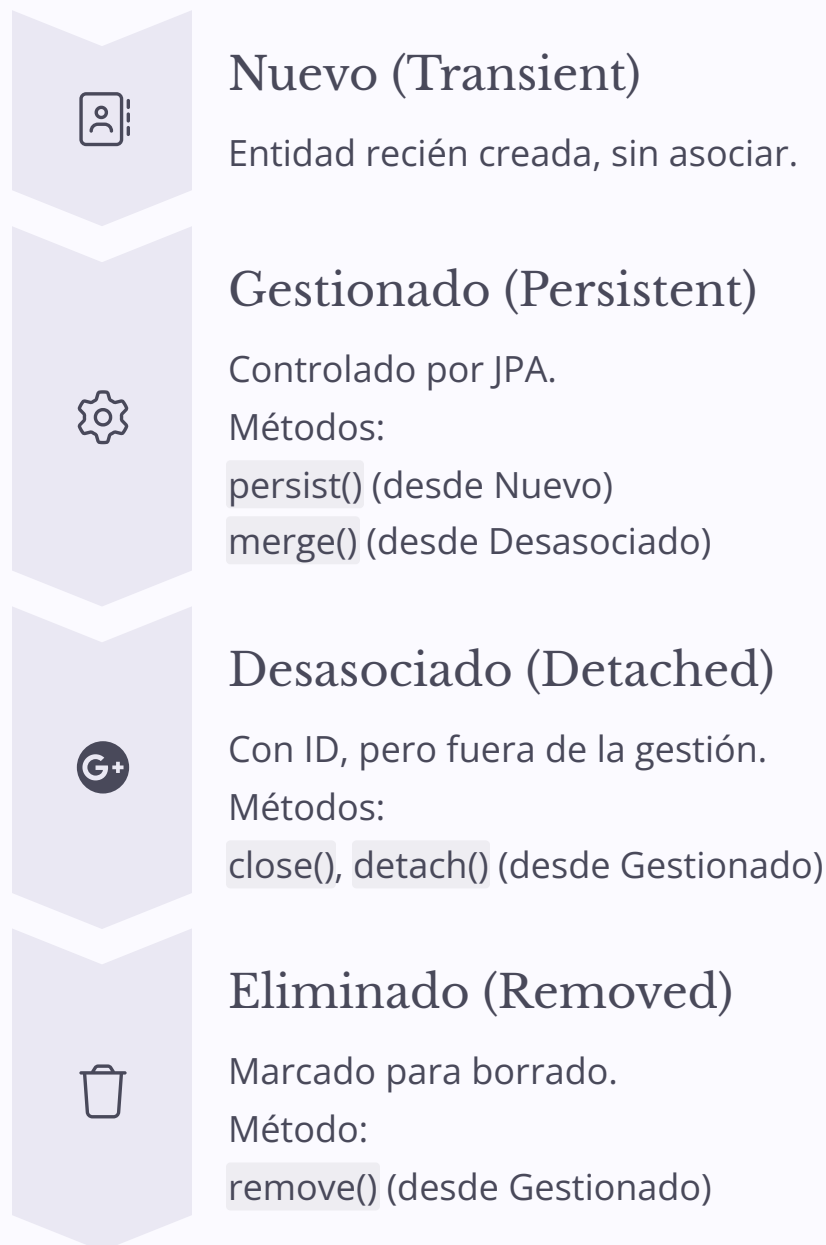
- Se transiciona desde **Gestionado** usando `remove()`.
- La eliminación física de la base de datos ocurre al hacer `commit` de la transacción.

Ejemplo

```
Producto p = em.find(Producto.class, 1L);  
em.remove(p);  
// 'p' está en estado Eliminado.  
// Se borra de DB al hacer commit.
```

Principales Transiciones en el Ciclo de Vida de una Entidad JPA

Las entidades se mueven entre estados mediante métodos específicos del `EntityManager`.



Este diagrama ilustra las principales rutas y los métodos que provocan cada cambio de estado.



Ejemplo Práctico Resumido

Un flujo común de estados de una entidad en una aplicación.

- Crear objeto: **NEW**
- Guardar con `persist()`: **MANAGED**
- Cerrar `EntityManager`: **DETACHED**
- Reasociar con `merge()`: **MANAGED**
- Eliminar con `remove()`: **REMOVED**

Buenas Prácticas con el Ciclo de Vida en JPA

Para asegurar el buen funcionamiento y rendimiento de tu aplicación, considera estas directrices:

✕ Cerrar el EntityManager

Siempre después de usarlo para liberar recursos.



Transacciones cortas

Mantenerlas breves para evitar bloqueos y mejorar escalabilidad.



Verificar estado

Asegurarse del estado de la entidad antes de cualquier operación.



Usar merge() solo si es necesario

Evitarlo cuando la entidad ya está gestionada.

Errores Comunes en el Ciclo de Vida de JPA

Comprender estos errores te ayudará a evitar problemas inesperados y a escribir código más robusto.



Modificar Entidad Desasociada

Realizar cambios en una entidad que ha sido desasociada del `EntityManager` no persistirá automáticamente en la base de datos. Necesitarás usar `merge()` para reasociarla y sincronizar los cambios.



LazyInitializationException

Intentar acceder a colecciones o relaciones cargadas de forma "lazy" (perezosa) después de que el `EntityManager` ha sido cerrado resultará en esta excepción. Asegúrate de inicializar todos los datos necesarios dentro de la sesión.



Olvido de commit()

Si no llamas a `entityManager.getTransaction().commit()` al final de una transacción, los cambios realizados (persistir, actualizar, eliminar) no se guardarán permanentemente en la base de datos, perdiéndose el trabajo.

Resumen del Ciclo de Vida en JPA

Una vista rápida de los estados y sus operaciones clave.

Nuevo (Transient)

Objeto recién instanciado.

No asociado al `EntityManager`.

Operación clave:

- `persist()`: Para pasar a Gestionado.

Gestionado (Persistent)

Bajo el control del `EntityManager`.

Cambios detectados automáticamente.

Operaciones clave:

- `find()`: Para recuperar.
- `merge()`: Para reasociar desde Desasociado.
- Acceso a propiedades: Sincroniza cambios.
- `remove()`: Para pasar a Eliminado.
- `detach()`: Para pasar a Desasociado.

Desasociado (Detached)

Tiene ID, pero fuera de gestión activa.

Cambios no se persisten automáticamente.

Operaciones clave:

- `merge()`: Para reasociar a Gestionado.
- Cierre de `EntityManager`, `detach()`.

Eliminado (Removed)

Marcado para ser borrado de la base de datos.

No disponible en el contexto de persistencia.

Operación clave:

- `remove()`: Marca para eliminación.
- `commit()` de la transacción: Elimina físicamente.

El `EntityManager` es quien decide en qué estado está tu entidad.

Próximos Pasos

¡Gracias!

En el próximo video vamos a hablar de **relaciones unidireccionales**, para aprender cómo conectar nuestras entidades.

